Search room results API

Comprehensive room search API using PostgreSQL pg_trgm fuzzy search with cursor-based pagination.

Jira Ticket

CONNECT-4605 - [SEARCH FEATURE REVAMP][HUBS-CMS-GO] add API for search rooms result DONE

Features

Fuzzy Search with pg_trgm

- Uses PostgreSQL's `pg_trgm` extension for fuzzy matching
- Searches both room title and description fields
- Multi-level boost scoring prioritizes different match types
- Supports multi-language search (Chinese, Japanese, Korean, etc.)
- Keywords automatically converted to lowercase for case-insensitive search
- Results ordered by: boost DESC, score DESC, room_id DESC

Cursor-Based Pagination (Keyset Pagination)

- Efficient pagination using cursor tokens
- No page offset issues with concurrent updates
- Cursor encodes: boost level, similarity score, and room ID
- Fetches `limit+1` results to determine if more pages exist
- `next_cursor` only included in response when more results are available
- 驗證pagination行為

Comprehensive Room Information

- Room ID, title, Hub SID
- Image URL (auto-converted to Directus asset URL)
- View count, like count
- Tags list (ordered by sort field)
- Owner name (fetched from community service)

- Preferred devices list

Community Integration

- Automatically queries owner community user names
- Only enabled when `ENABLE_FEATURE_COMMUNITY=true`
- Batch query optimization to avoid N+1 problem

Search History Tracking

- Automatically saves search keywords to user's recent search
- Only for authenticated users (requires HTC account ID)
- Configurable max items limit (default: 3)
- Runs asynchronously to not block response

API Specification

Request

Method: `POST`

Endpoint: `/api/hubs-cms/v2/rooms/search`

Content-Type: `application/json`

Request Body:

Parameter	Type	Required	Description
q	string	Yes	Search keyword (2-50 characters)
cursor	string	No	Pagination cursor from previous response
limit	int	No	Results per page (1-100, default: 20)
sort	string	No	most_viewed (default), most_liked

			, create_date, first_public_date
device	string	No	Filter by preferred device type(e.g., “ `desktop` ")
tag	string	No	Filter by tags, comma-separated for multiple (e.g., "Hangout,AI")

Request Example (First Page):

```
{
  "q": "demo room",
  "sort": "most_viewed",
  "tag": "Art",
  "device": "desktop",
  "limit": 20
}
```

Request Example (Subsequent Page):

```
{
  "q": "demo room",
  "sort": "most_viewed",
  "tag": "Art",
  "device": "desktop",
  "cursor": "eyJib29zdCI6NSwic2NvcmUiOjAuMzMzMzMzMzLCJyb29tX2lkIjoiMTIzIn0=",
  "limit": 20
}
```

Response

Status Code: `200 OK`

Response Body:

Field	Type	Required	Description
results	SearchRoomItem[]	Yes	Array of room search results

next_cursor	string	No	Cursor for next page (only present if more results exist)

SearchRoomItem Fields:

Field	Type	Description
id	string	Room ID
title	string	Room title
hub_sid	string	Hub SID (may be empty string)
image	string	Full image URL (auto-converted to Directus asset URL)
view_count	number	View count
like_count	number	Like count
tags	string[]	Tags list (ordered by sort field)
owner_name	string	Owner name (queried from community service, may be empty)
preferred_devices	string[]	Preferred devices list
is_liked	bool	Whether current user has liked this room

Response Examples

Success - With More Results:


```
{
  "next_cursor": "eyJib29zdCI6NSwic2NvcmUiOjAuMzMzMzMzMzLCJyb29tX2lkIjoiNDU2In0=",
  "results": [
    {
```

```
      "id": "123",
      "title": "Demo Room",
      "hub_sid": "abc123",
      "image": "https://directus.example.com/assets/image-id",
      "view_count": 100,
      "like_count": 25,
      "tags": ["VR", "Social"],
      "owner_name": "John Doe",
      "preferred_devices": ["Quest", "Desktop"]
    },
    {
      "id": "456",
      "title": "Another Demo Room",
      "hub_sid": "def456",
      "image": "https://directus.example.com/assets/image-id-2",
      "view_count": 50,
      "like_count": 10,
      "tags": ["Game", "VR"],
      "owner_name": "Jane Smith",
      "preferred_devices": ["Quest"]
    }
  ]
}
```

Success - Last Page (No More Results):

```
{
  "results": [
    {
      "id": "789",
      "title": "Final Demo Room",
      "hub_sid": "ghi789",
      "image": "https://directus.example.com/assets/image-id-3",
      "view_count": 30,
      "like_count": 5,
      "tags": ["Education"],
      "owner_name": "Bob Wilson",
      "preferred_devices": ["Desktop"]
    }
  ]
}
```

Note: `next_cursor` is omitted when there are no more results.

Success - 204 No Content (No Matches):

When no rooms match the search criteria, the API returns a

```
204 No Content
```

status with no response body.

Error - 400 Bad Request (Invalid Request Format or Cursor):

```
{
  "error": {
    "code": 400400,
```

```
    "message": "Invalid param: request param"
  }
}
```

Error - 500 Internal Server Error:

```
{
  "error": {
    "code": 500,
    "message": "Internal error"
  }
}
```

Response Pagination Rules

How Pagination Works

1. **Request Processing:**
   - API fetches `limit + 1` results from database
   - If result count > `limit`, more pages exist
2. **Response Generation:**
   - Returns exactly `limit` items in `results` array (or fewer on last page)
   - If more results exist: includes `next_cursor` in response
   - If no more results: omits `next_cursor` field entirely
3. **Cursor Behavior:**
   - **First page:** Don't include `cursor` parameter in request
   - **Subsequent pages:** Use `next_cursor` from previous response as `cursor` parameter
   - **Last page detection:** Response without `next_cursor` field indicates last page
   - **Empty results:** When `results` is empty array, no `next_cursor` is included
4. **Cursor Structure:**
   - Base64-encoded JSON containing: `{boost, score, room_id}`
   - Example decoded: `{"boost":5,"score":0.333333,"room_id":"456"}`
   - Represents the last item from the current page
   - Next query fetches items where `(boost, score, room_id) < (cursor values)`

Example Pagination Flow

```
Request 1: {"q": "demo", "limit": 20}
Response 1: {
  "next_cursor": "eyJ...=",  ← More results exist
  "results": [20 items]
}

Request 2: {"q": "demo", "cursor": "eyJ...=", "limit": 20}
Response 2: {
  "next_cursor": "eyJ...=",  ← More results exist
  "results": [20 items]
}

Request 3: {"q": "demo", "cursor": "eyJ...=", "limit": 20}
Response 3: {
  "results": [15 items]  ← No next_cursor = last page
}
```

Query Validation Rules

All search queries are validated for security:

Requirements

- **Minimum length:** 2 characters
- **Maximum length:** 50 characters
- **Allowed characters:** All printable characters and Unicode text (supports multi-language)
- **Rejected characters:** Control characters only (e.g., `\\0`, `\\n`, `\\r`, tab, etc.)

Valid Queries

✅ `hello world`

✅ `Harry Potter`

✅ `what's up?`

✅ `test-123`

✅ `こんにちは` (Japanese)

✅ `你好` (Chinese)

✅ `안녕하세요` (Korean)

Invalid Queries

❌ `a` (too short - minimum 2 characters)

❌ `test\\nroom` (contains control character `\\n`)

❌ `room\\0test` (contains null character)

Search Algorithm

1. Keyword Processing

- Trim leading/trailing whitespace
- Convert to lowercase for case-insensitive search

2. Boost Scoring System

Results are ranked by **boost level** (primary) and **similarity score** (secondary):

Match Type	Boost Level	Example (keyword: "demo")
Title prefix match	6	"**Demo** Room" (starts with "demo")
Title fuzzy match (pg_trgm)	5	"Demonstration" (similar to "demo")
Title infix match	4	"My **Demo** World" (contains "demo")
Description prefix match	3	Description starts with "demo"
Description infix match	2	Description contains "demo"
Description fuzzy match	1	Description similar to "demo"

3. Similarity Score Calculation

- Uses PostgreSQL `similarity()` function from pg_trgm extension
- Formula: `MAX(title_similarity, description_similarity * 0.8)`
- Rounded to 6 decimal places
- Range: 0.0 to 1.0 (higher is better match)

4. Result Ordering

Results are sorted by:

1. **boost** (DESC) - Higher boost levels appear first
2. **score** (DESC) - Higher similarity scores appear first

3. **room_id** (DESC) - Consistent ordering for pagination

5. Owner Names Integration

- Collects unique owner HTC account IDs from results
- Batch fetches community user names (if `ENABLE_FEATURE_COMMUNITY=true`)
- Gracefully handles community service errors (empty owner_name if unavailable)

6. Search History

- Saved asynchronously after response is sent
- Only for authenticated users (requires HTC account ID header)
- Deduplicates existing keywords (doesn't save duplicates)
- Prepends new keyword to beginning of list
- Limits to `RECENT_SEARCH_MAX_ITEMS` (default: 3)

Environment Variables

Community Integration

Variable	Default	Description
ENABLE_FEATURE_COMMUNITY	false	Enable community user name lookup

Database Configuration

Variable	Default	Description
DATABASE_HOST		PostgreSQL host (required)
DATABASE_PORT	5432	PostgreSQL port
DATABASE_USER		PostgreSQL user
DATABASE_PASSWORD		PostgreSQL password
DATABASE_NAME		PostgreSQL database name
DATABASE_SSL_MODE	disable	PostgreSQL SSL mode

DATABASE_MAX_CONNS	10	Max pool connections
DATABASE_MIN_CONNS	2	Min pool connections
DATABASE_MAX_CONN_LIFETIME	1h	Connection max lifetime
DATABASE_MAX_CONN_IDLE_TIME	30m	Connection max idle time

Example Usage

Paginating Through All Results

```
# First page
curl -X POST https://api.example.com/api/hubs-cms/v2/rooms/search \
  -H "Content-Type: application/json" \
  -d '{"q": "vr room", "limit": 10}'

# Response includes next_cursor if more results exist
# {
#   "next_cursor": "eyJib29zdCI6NSwic2NvcmUiOjAuNSwicm9vbV9pZCI6IjEwIn0=",
#   "results": [...]
# }

# Second page - use cursor from previous response
curl -X POST https://api.example.com/api/hubs-cms/v2/rooms/search \
  -H "Content-Type: application/json" \
  -d '{
    "q": "vr room",
    "cursor": "eyJib29zdCI6NSwic2NvcmUiOjAuNSwicm9vbV9pZCI6IjEwIn0=",
    "limit": 10
  }'

# Continue until response has no next_cursor field
```

Detecting Last Page

```
async function getAllResults(keyword) {
  const allResults = [];
  let cursor = null;

  while (true) {
    const response = await fetch('/api/hubs-cms/v2/rooms/search', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        q: keyword,
        cursor: cursor,
```

```
        limit: 20
      })
    });

    const data = await response.json();
    allResults.push(...data.results);

    // Check if more results exist
    if (!data.next_cursor) {
      break; // Last page reached
    }

    cursor = data.next_cursor;
  }

  return allResults;
}
```